

Design and Implementation of a 5-bit Carry Lookahead Adder in 180 nm CMOS Technology

Satvik Shrivastava
satvik.shrivastava@students.iit.ac.in
IIIT Hyderabad
2024102029

I. INTRODUCTION

Design of a 5-bit Carry Look-Ahead (CLA) Adder using static CMOS and TSPC logic has been presented. Performance parameters of the proposed 5-bit CLA architecture have been simulated validated by designing a layout and extracting parameters to conduct spice simulations. MAGIC layout design tool and NGSpice spice circuit simulator engine was used to extract design parameters and performs simulations for 180 nm technology. The design was analyzed in terms of average power consumption, propagation delay and power delay product (PDP).

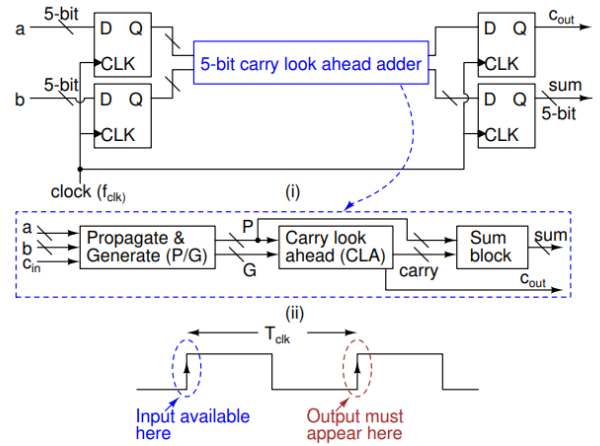


Fig. 1 Basic Design of a 5-bit CLA

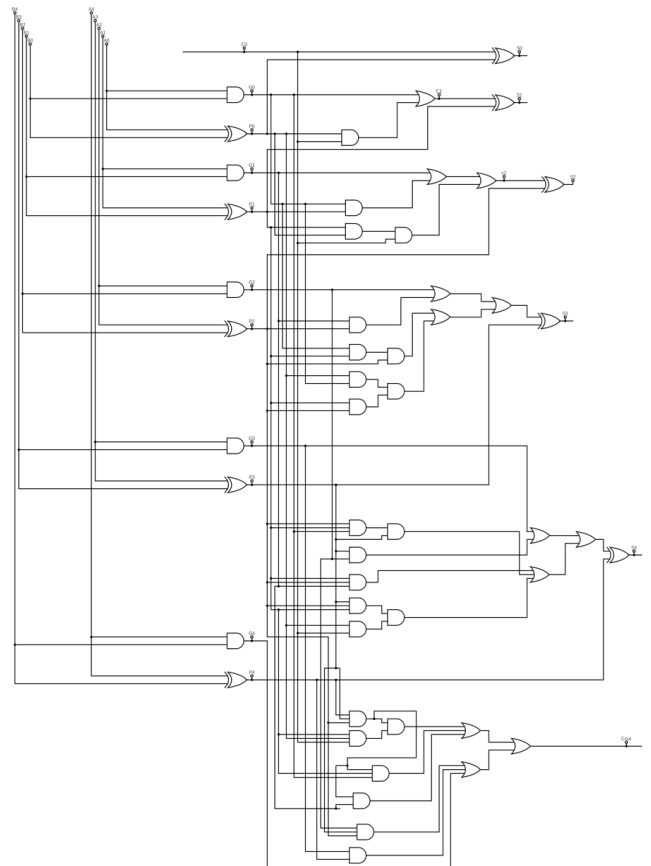
II. PROPOSED DESIGN

The proposed 5-bit carry look-ahead adder (CLA) is designed to meet high-speed performance requirements while ensuring accurate functionality in synchronous operation. The primary objective of the design is to compute the output sum and carry within one clock cycle, ensuring that the output is valid at the next rising edge of the clock after the inputs are latched. The detailed analysis and the exact design of each block are presented in the design methodology subsection

The CLA adder is designed using a modular approach, consisting of the following blocks:

- D-Flip-Flops: To latch inputs and intermediate signals, ensuring synchronization with the clock.
- Propagate and Generate Logic: For computing prop gate ($p_i = a_i \oplus b_i$) and generate ($g_i = a_i \cdot b_i$) signals.
- Carry Computation Logic: To compute carry signals in parallel, minimizing the delay associated with sequential carry propagation.
- Sum Logic: To compute the prepare Your Paper Before Styling

Each block is implemented with carefully chosen logic styles and transistor sizing to optimize performance in terms of speed, power, and area.



III. D FLIP FLOP

Functionality :

A positive edge-triggered D flip-flop is a sequential digital circuit that transfers the data input (D) to its output (Q) only during the rising edge of the clock signal. It is commonly used in synchronous systems for data storage, synchronization, and timing control.

- **Edge-Sensitivity:** The flip-flop responds only during the positive edge (rising edge) of the clock signal, i.e., the transition from 0 to 1.
- **Data Storage:** The value at the input D is sampled at the rising edge of the clock and stored internally. This value appears at the output Q and remains stable until the next rising edge.
- **Output Behavior:**– If D = 1 at the positive edge, Q becomes 1.– If D = 0 at the positive edge, Q becomes 0

clock	D	Q(n+1)
1	0	0
1	1	1
0	X	Q(n)

D-Flip-Flop Design

Transistor sizing is done with respect to the minimum-sized 2W/W inverter.

The width of the PMOS and NMOS of the minimum-sized inverter is $W = 20\lambda$, where $\lambda = 0.09\mu\text{m}$. Thus, the sizes are:

- P1, P2, P3, P7 have a sizing of 4W each
- N1 and N6 have a sizing of W each
- N2, N3, N4, N5, P5, P6 have a sizing of 2W each

NGSPICE Stimulation:

Inputs are provided to Flip flop as follows

A Clock signal of period 10ns with Rise time and Fall time of 0s A Input signal is delayed.

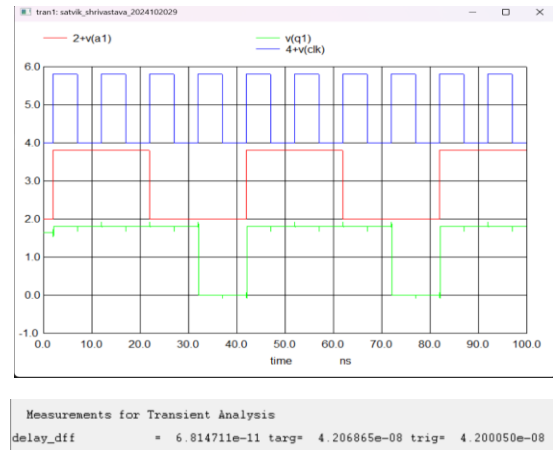


Fig. 4: D-flipflop output waveform

GATES:

2 Input AND Gate

The following truth table shows the AND for all possible combinations of inputs a and b.

a	b	Y=a.b
0	0	0
0	1	0
1	0	0
1	1	1

Size of PMOS of the NAND Gate: (40*λ)

Size of NMOS of the NAND Gate: (40*λ)

3 Input AND Gate

The truth table displays the AND operation for all possible combinations of inputs a, b and c:

a	b	c	Y=a.b.c
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

A 3-Input AND Gate can be designed using Static CMOS Logic, implemented with a Static CMOS 3-input NAND gate followed by a Static CMOS inverter. To achieve delay comparable to a Minimum Sized Inverter, the components are sized as:

- Size of PMOS of NAND Gate: (40 * λ)
- Size of NMOS of NAND Gate: (60 * λ)

4 Input AND Gate

The following truth table shows the AND for all possible combinations of inputs a, b, c and d:

a	b	c	d	Y=a.b.c.d
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

A 4 Input AND Gate can be designed using Static CMOS Logic . It is designed to built using StaticCMOS4-input NAND gate and a Static CMOS Inverter. 4-input NAND gate to have delay close to Delay of Minimum Sized Inverter it is sized as follow:

Size of PMOS of NAND Gate: $(40*\lambda)$

Size of NMOS of NAND Gate: $(80*\lambda)$

5 Input AND Gate

The truth table displays the AND operation for all possible combinations of inputs a, b and c:

a	b	c	d	e	Y = a.b.c.d.e
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	0
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	0	1	0
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	1

A 5 Input AND Gate can be designed using Static CMOS Logic . It is designed to built using StaticCMOS4-input NAND gate and a Static CMOS Inverter. 5-input NAND gate to have delay close to Delay of Minimum Sized Inverter it is sized as follow:

Size of PMOS of NAND Gate: $(40*\lambda)$

Size of NMOS of NAND Gate: $(100*\lambda)$

2 Input OR Gate

The following truth table shows the OR for all possible combinations of inputs a and b:

a	b	Y=a+b
0	0	0
0	1	1
1	0	1
1	1	1

A 2 Input OR Gate can be designed using Static CMOS Logic . It is designed to built using Static CMOS 2-input NOR gate and a Static CMOS Inverter. 2-input NOR gate to have delay close to Delay of Minimum Sized Inverter it is sized as follow :

Size of PMOS of NOR Gate: $(80*\lambda)$

Size of NMOS of NOR Gate: $(20*\lambda)$

3 Input OR Gate

The following truth table shows the OR for all possible combinations of inputs a, b and c:

a	b	c	Y=a+b+c
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

A-3 Input OR Gate can be designed using Static CMOS Logic . It is designed to build using Static CMOS 3-input NOR gate and a Static CMOS Inverter .3-input Norgate to have delay close to Delay of Minimum Sized Inverter it is sized as follow :

Size of PMOS of NOR Gate: $(120*\lambda)$

Size of NMOS of NOR Gate: $(20*\lambda)$

4 Input OR Gate

The following truth table shows the AND for all possible combinations of inputs a, b, c and d:

A	b	c	d	$Y=a+b+c+d$
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

A-4 Input OR Gate can be designed using Static CMOS Logic . It is designed to built using Static CMOS 4-input NOR gate and a Static CMOS Inverter .4-input NOR gate to have delay close to Delay of Minimum Sized Inverter it is sized as follow :

Size of PMOS of NOR Gate: $(160*\lambda)$

Size of NMOS of NOR Gate: $(20*\lambda)$

5 Input OR Gate

The following truth table shows the AND for all possible combinations of inputs a, b, c and d:

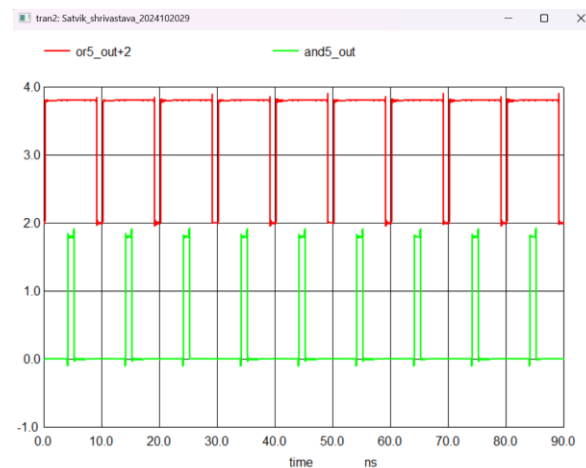
a	b	c	d	e	$Y = a+b+c+d+e$
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	1
0	0	1	0	0	1
0	0	1	0	1	1
0	0	1	1	0	1
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	1
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	1
1	1	1	0	0	1
1	1	1	0	1	1
1	1	1	1	0	1
1	1	1	1	1	1

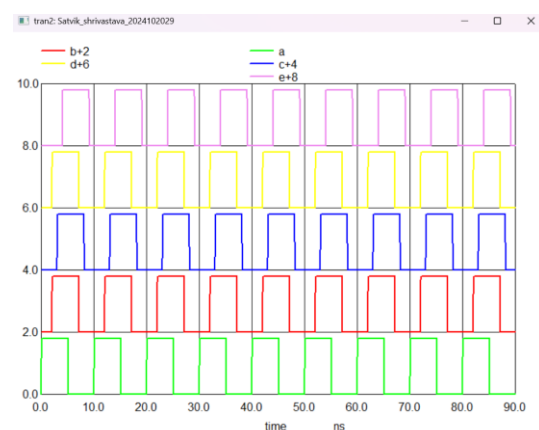
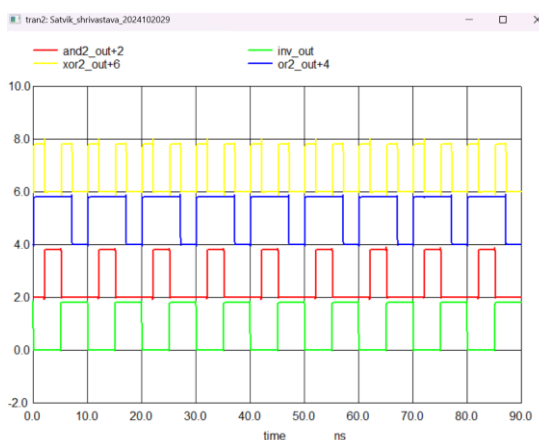
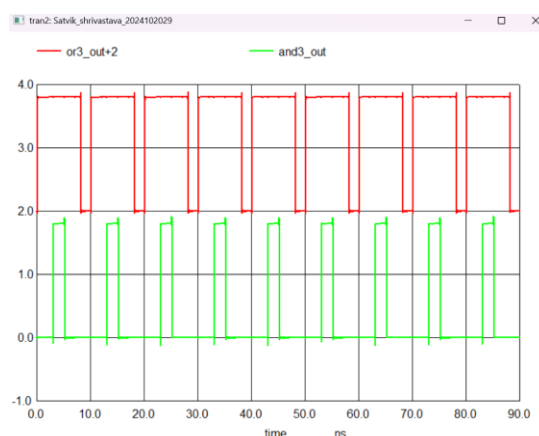
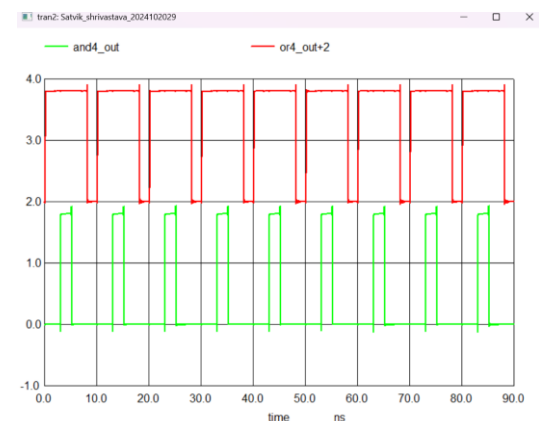
A-5 Input OR Gate can be designed using Static CMOS Logic . It is designed to built using Static CMOS 5-input NOR gate and a Static CMOS Inverter .5-input NOR gate to have delay close to Delay of Minimum Sized Inverter it is sized as follow :

Size of PMOS of NOR Gate: $(200*\lambda)$

Size of NMOS of NOR Gate: $(20*\lambda)$

Plots:





Propagate and Generate BLOCK

Functionality:

The Propagation(p) and Generate(g) signal are defined as follows:

The Propagation signal indicates whether the Carry-in (cin) will propagate to the Carry-out (cout): $p = a \oplus b$

•When $p=1$, any incoming Carry(cin) will propagate to the next stage.

•When $p=0$, the Carry-in is blocked from propagating. The generate signal indicates whether a Carry is generated by the addition of the inputs a and b, irrespective of the Carry-in: $g = a \cdot b$.

•When $g=1$, a carry is generated in the current stage and when $g=0$, no carry is generated by the current stage. These signals are widely used in carry look-ahead adders (CLA) for efficient carry computation, reducing the delay caused by the Ripple-carry effect in traditional adders. The following truth table shows the values of propagation(p) and generate(g) signals for all possible combinations of inputs a and b

a	b	$P = a \oplus b$	$G = a \cdot b$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

CLA BLOCK

Functionality:

Carry Look-Ahead Adder (CLA) Characteristics:

Propagate (Pi) and Generate (Gi) Signals:

$P_i = a_i \oplus b_i$ (Carry propagates if inputs differ)

$G_i = a_i \cdot b_i$ (Carry is generated if both inputs are 1) Carry Computation (parallel):

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$C_5 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 G_0 + P_4 P_3 P_2 P_1 P_0 C_0$$

Delay Analysis: Proportional to $\log(n)$
where n = number of bits

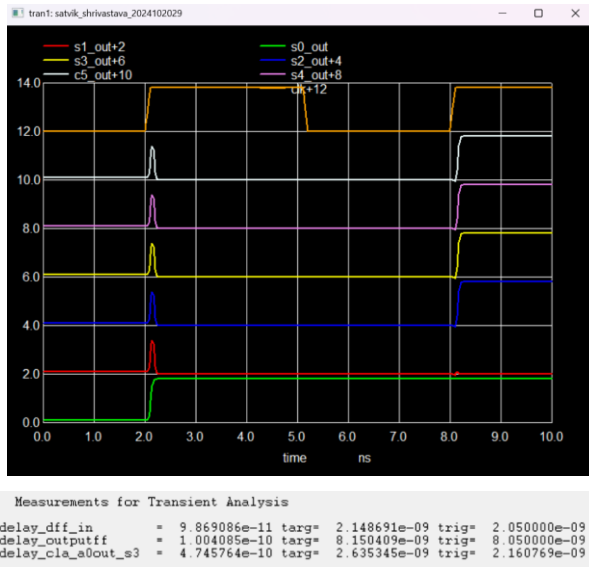
Advantages:

- 1) Faster than RCA for large bit-widths (parallel computation)
- 2) Suitable for high-speed designs

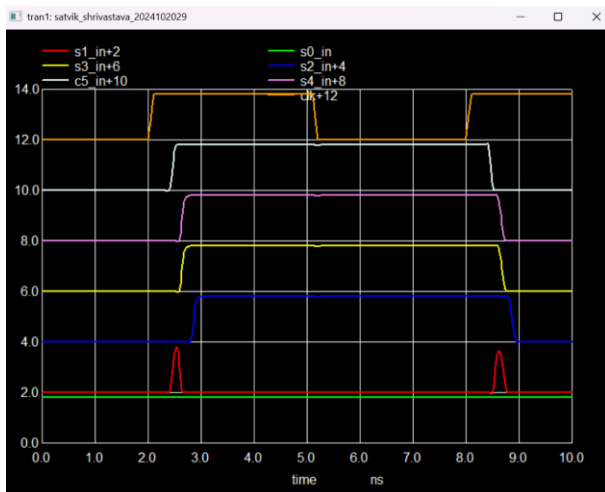
Disadvantages:

- 1) More complex circuit design
- 2) Higher power consumption and area

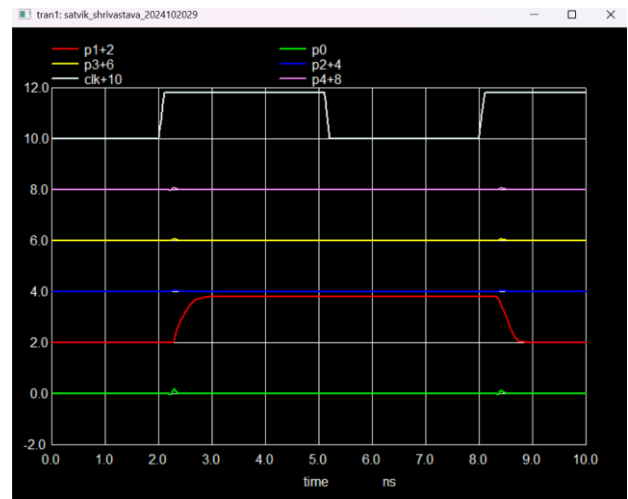
FINAL CIRCUIT



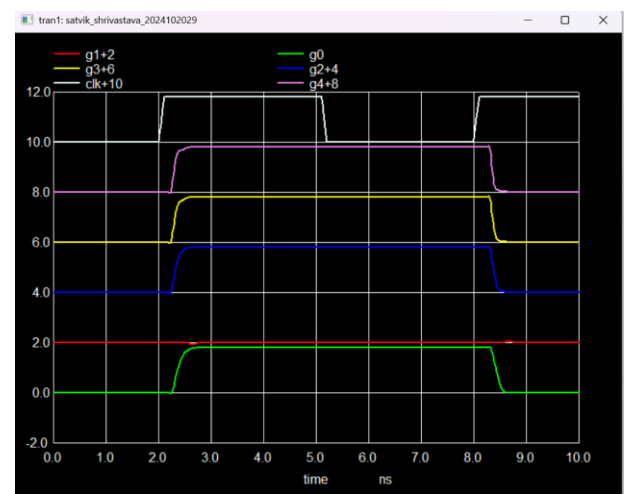
Final sum



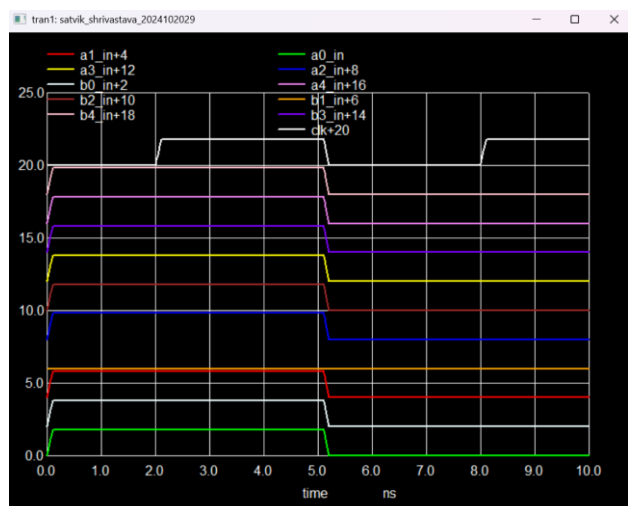
OUTPUTS SUM AND C4 BEFORE OUTPUT FLIPFLOP



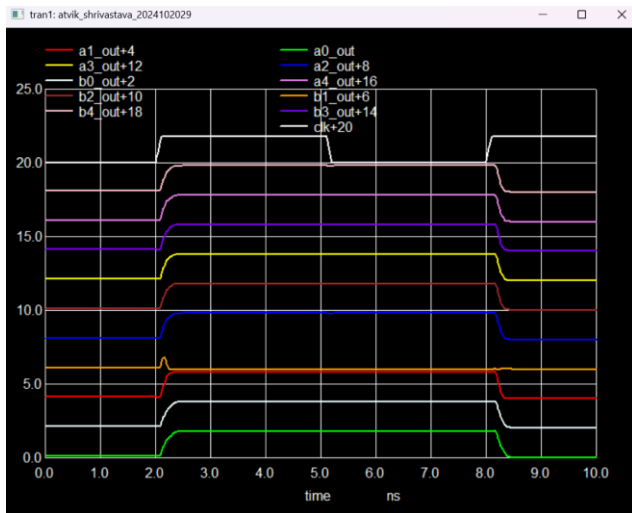
PROPAGATES



GENERATES



Input set of 4-Bits A , B before d flip flop



Input set of 4-Bits A , B after d flip flop

Inputs given:

A4 A3 A2 A1 A0 = 11111

B4 B3 B2 B1 B0 = 11101

Expected Output:

S5 S4 S3 S1 S0 = 11100

c5 = 1

CALCULATIONS

$$T_{CLK} \geq T_{SU} + T_{PCQ} + T_p$$

$$T_{CLK} \geq T_{su} + 100.41 \text{ ps} + 474.58 \text{ ps}$$

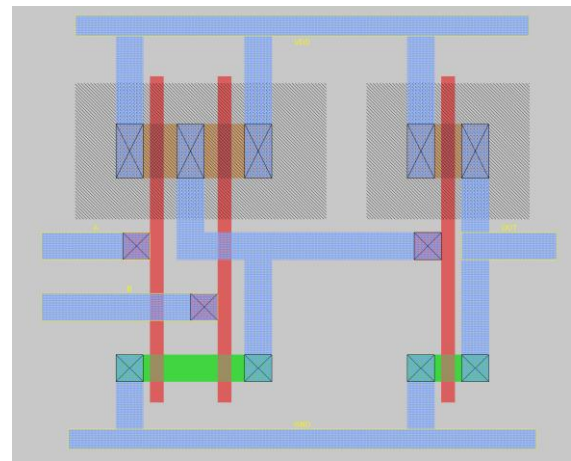
$$T_{CLK} \geq 574.99 \text{ ps} + T_{su}$$

Post Layout:

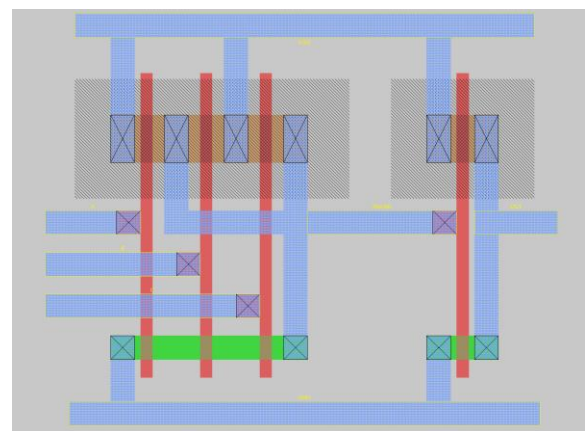
$$T_{CLK} \geq T_{SU} + T_{PCQ} + T_p$$

$$T_{CLK} = T_{su} + 250 + 759$$

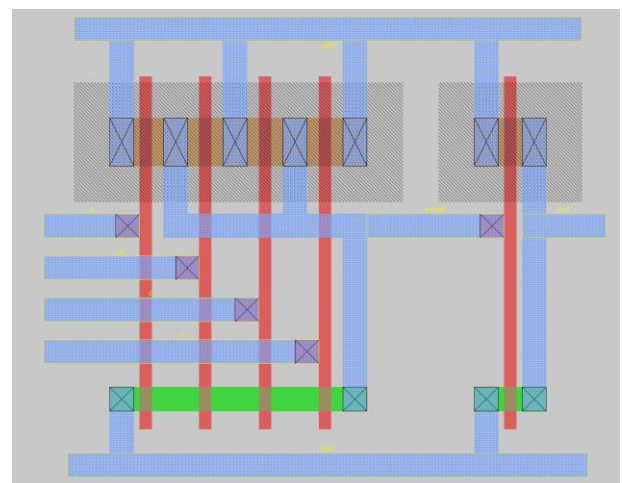
Magic:



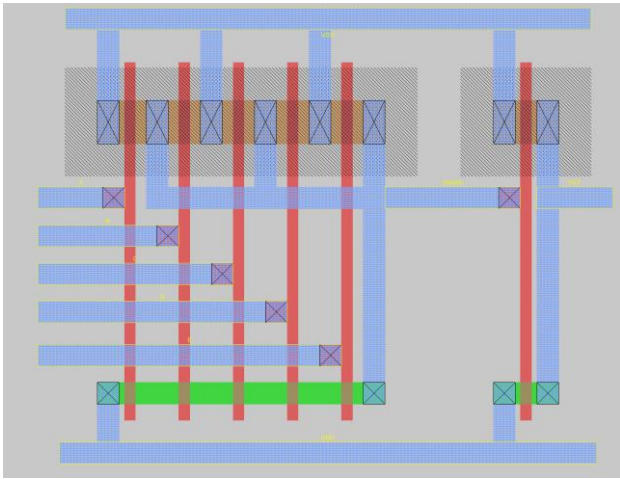
2i/p AND



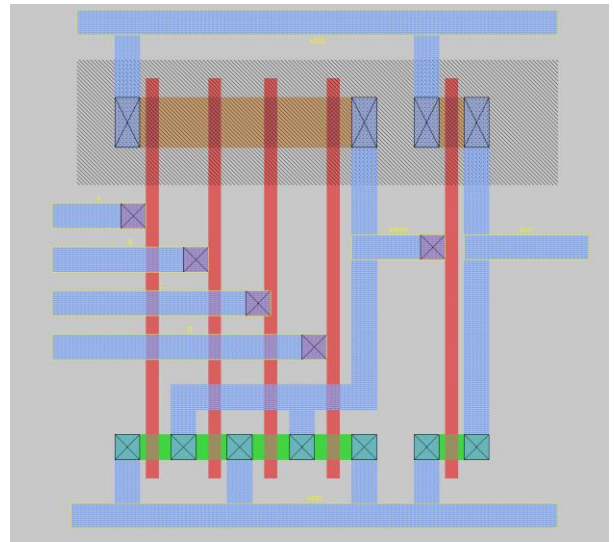
3 i/p AND



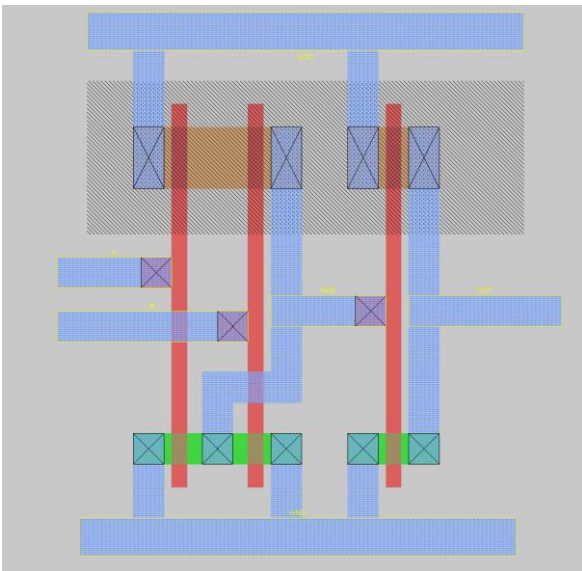
4 i/p AND



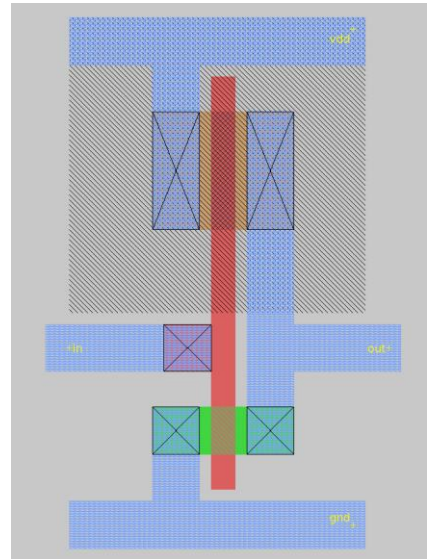
5i/p AND



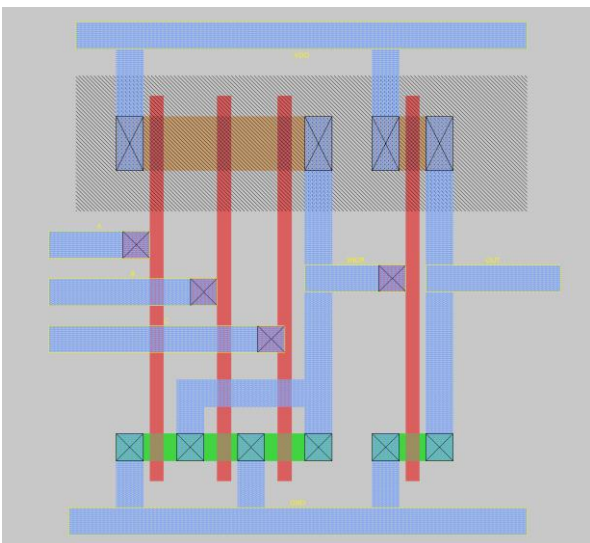
4 I/P OR



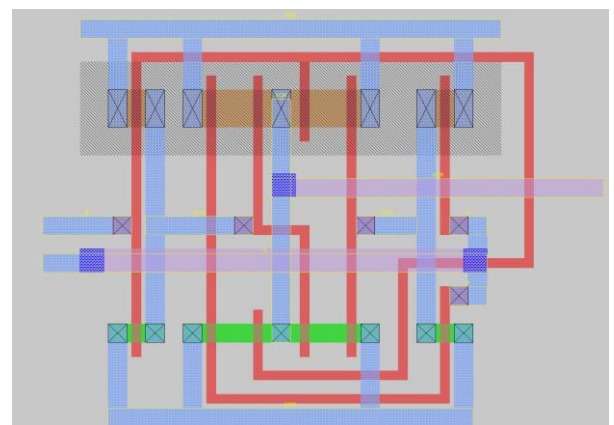
2 i/p OR



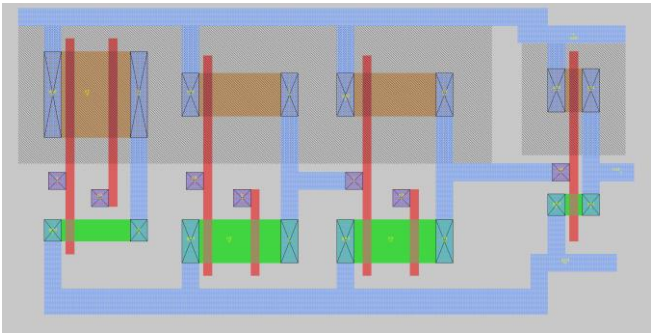
Inverter



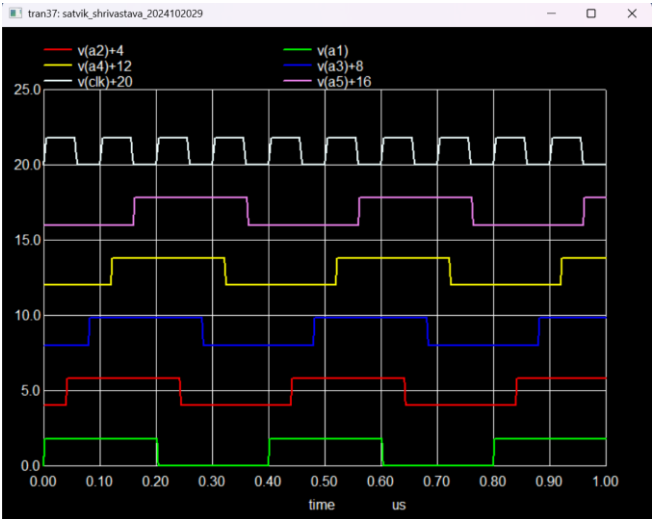
3 I/P OR



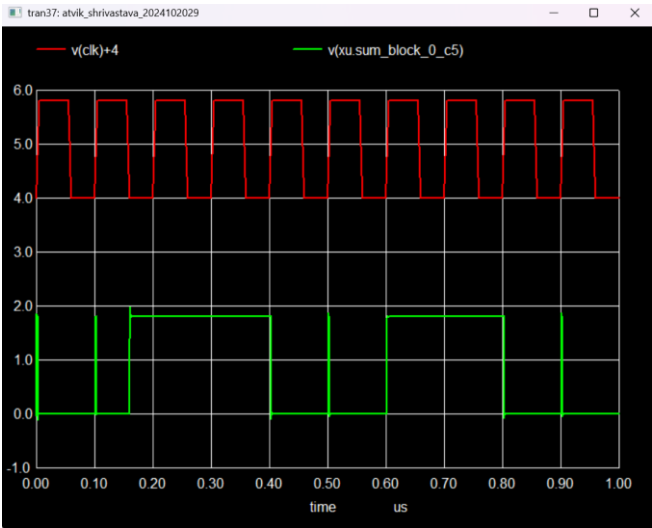
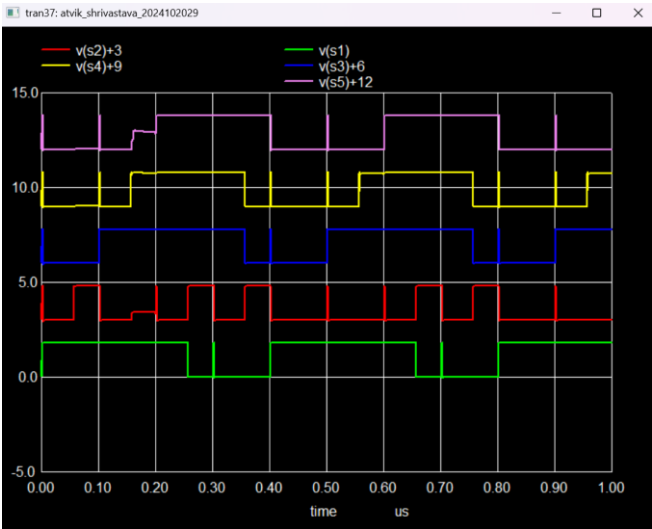
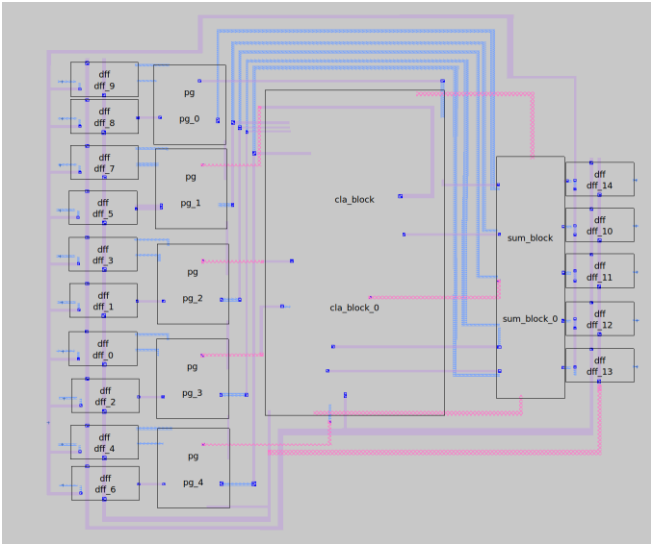
2 I/P XOR



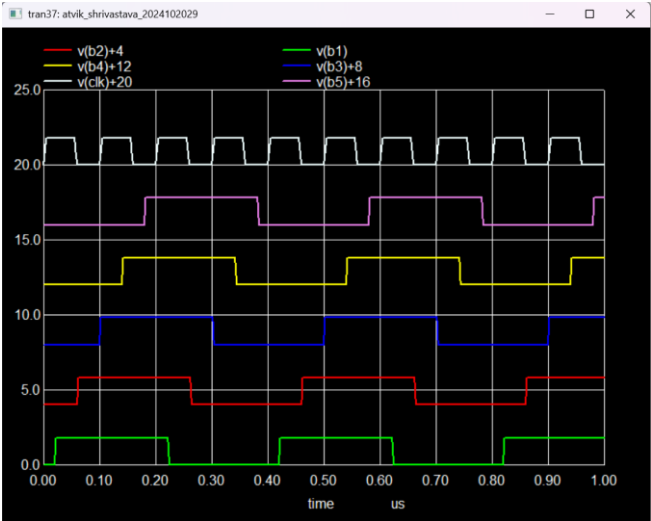
D Flip Flop



Complete Layout

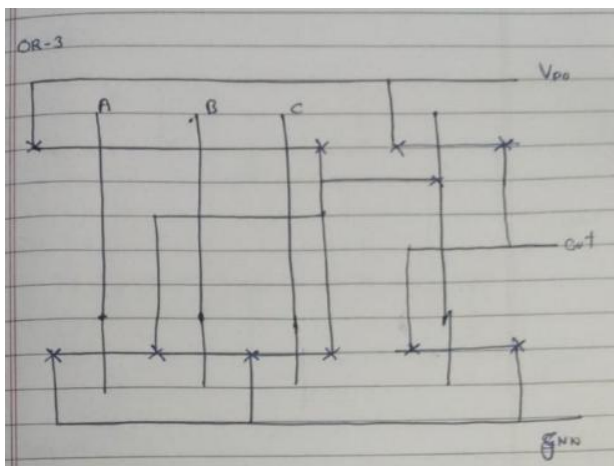
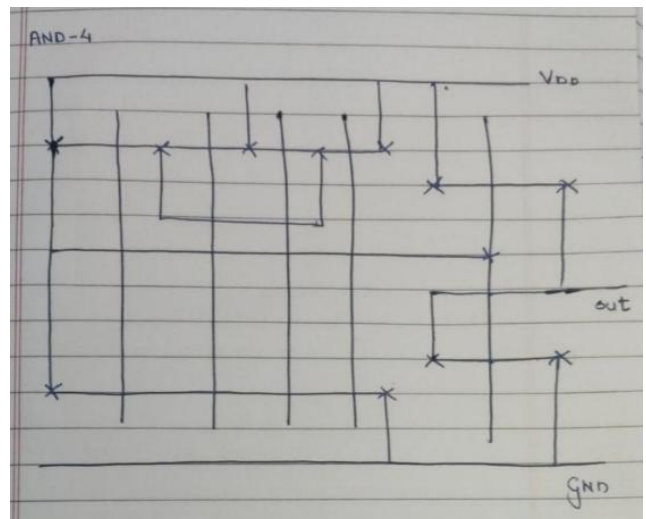
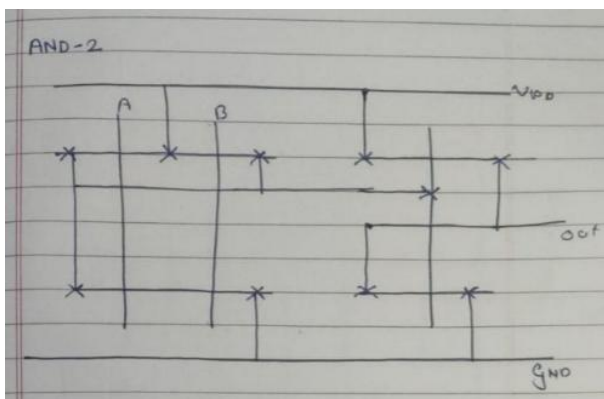
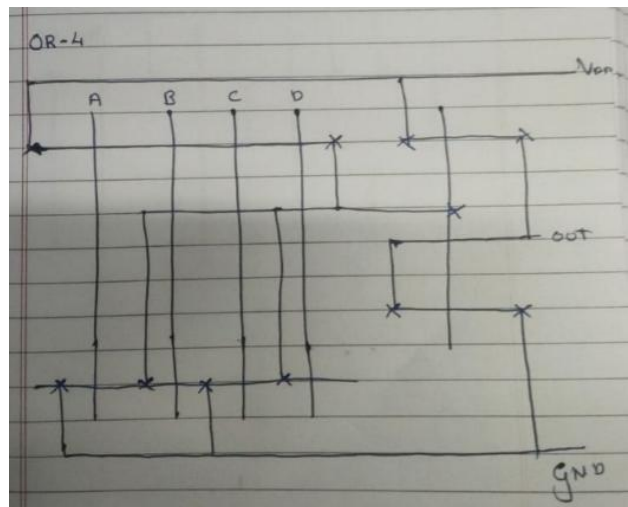
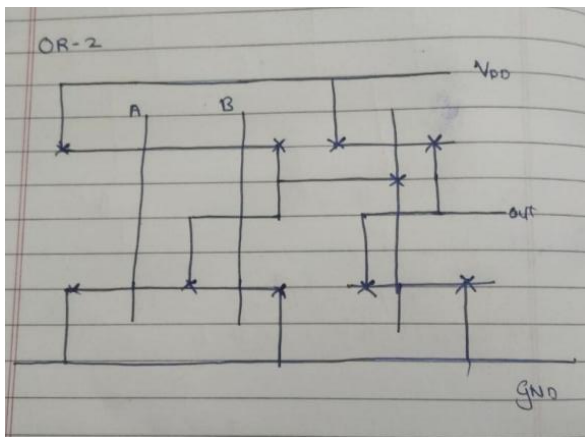
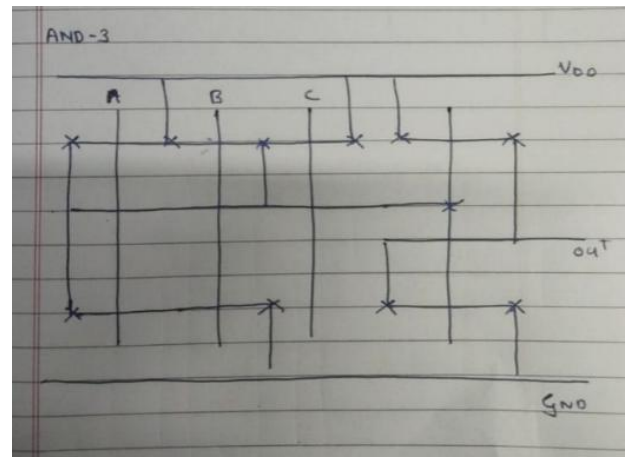
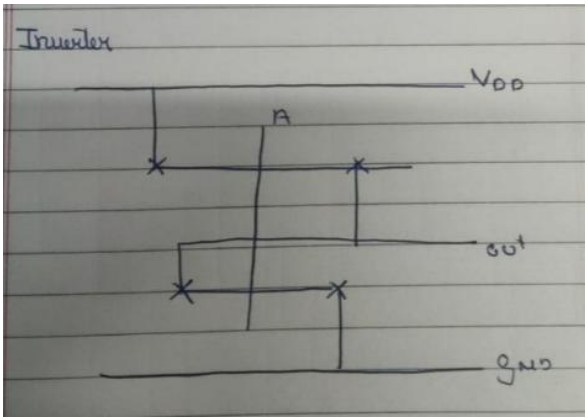


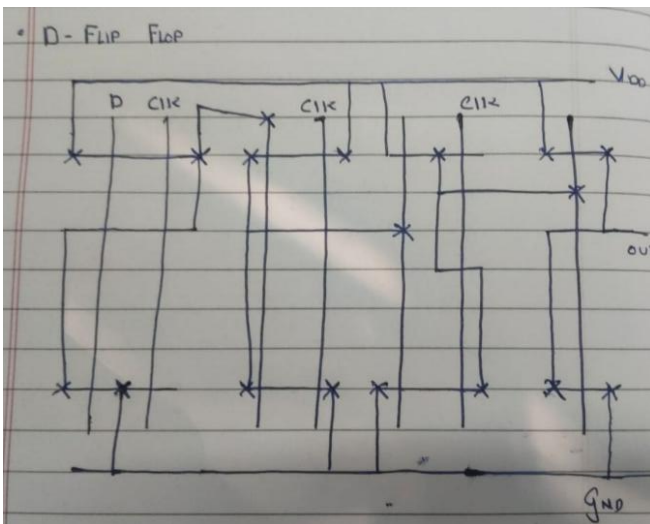
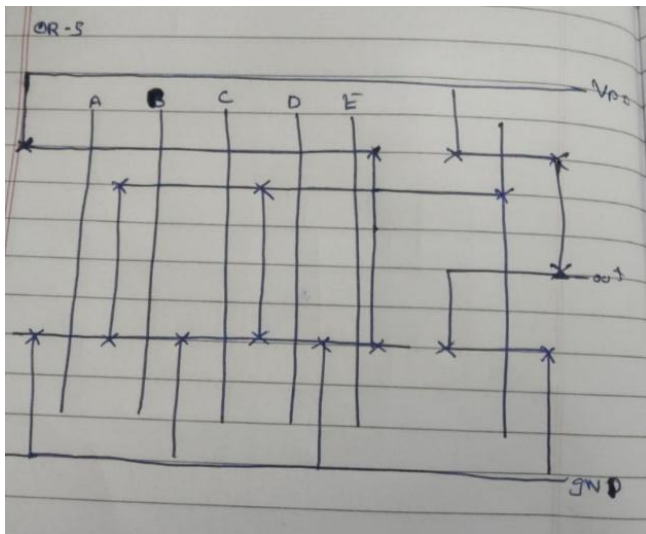
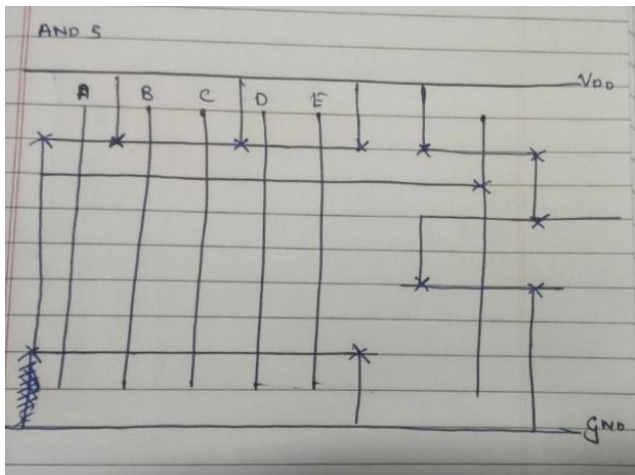
Post Layout Simulation:



Measurements for Transient Analysis	
<code>t_a1</code>	<code>= 1.00000e-09</code>
<code>t_s1</code>	<code>= 1.75913e-09</code>
<code>t_cout</code>	<code>= 1.98366e-09</code>
<code>t_clk_edge</code>	<code>= 2.50000e-09</code>
<code>t_q_launch</code>	<code>= 1.74651e-09</code>
<code>t_d_capture</code>	<code>= 1.75913e-09</code>
<code>t_q_capture</code>	<code>= 2.03498e-09</code>

STICK DIAGRAMS





VERILOG HDL

This code describes a 5-bit Carry-Lookahead Adder (CLA) implemented in Verilog.

The design uses flip-flops for input and output registers, making it a pipelined version of the adder.

The CLA architecture improves addition speed by precomputing carry signals, unlike simpler ripple-carry adders, which must wait for each bit's carry to propagate sequentially

Input and Output Declarations:

- Inputs: Two 5-bit numbers (a and b), and a clock signal (clk).
- Outputs: 5-bit sum (sum) and a carryout bit (cout).

Register Declarations:

Temporary registers hold input values after being latched by flip-flops, ensuring stable signals during addition.

Flip-Flop Instantiation:

Each bit of inputs a and b is latched using D flip-flops (dff). This captures the inputs on each rising clock edge, ensuring synchronization.

Generate (g) and Propagate (p) Signals:

- Generate (g): Indicates whether a carry is generated at a bit position.
- Propagate (p): Indicates whether a carry will propagate from the previous position.

Carry Calculation (c):

- The carry logic uses CLA principles to compute carries in parallel, reducing delay:

$c[1]$ depends on $g[0]$ and $p[0]$.

$c[2]$ depends on $g[1]$, $p[1]$, and previous carries, and so on up to $c[4]$.

And so on

Sum Calculation:

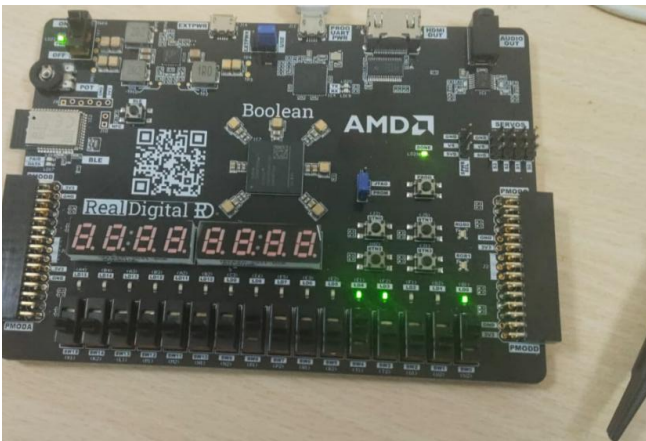
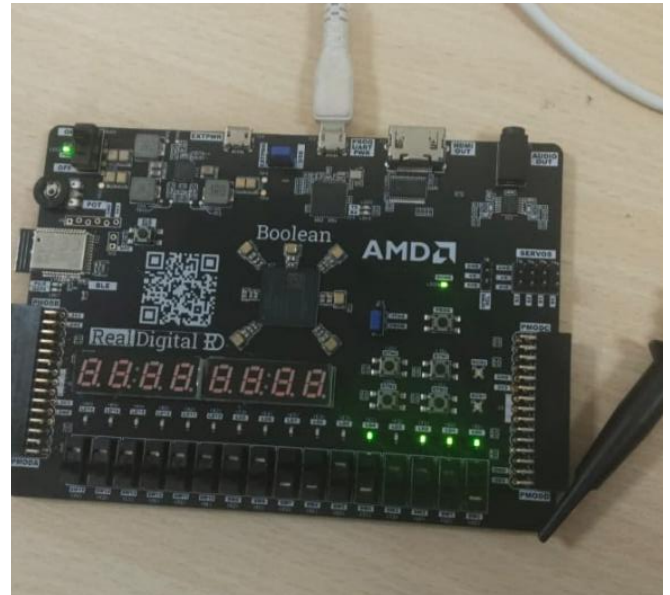
The sum for each bit is calculated by XOR-ing the corresponding p and c values.

Summary:

This pipelined CLA adder improves speed and efficiency for 4-bit addition by leveraging carry lookahead logic. Input and output registers provide stability and synchronization, crucial for high-speed digital systems.



VERILOG IMPLEMENTATION AND VERIFICATION WITH OSCILLOSCOPE



REFERENCES

- M. Hasan, A. H. Siddique, A. H. Mondol, M. Hossain
- H. U. Zaman, and S. Islam, "High-performance design of a 4-bit carry look-ahead adder in static CMOS logic,"
- J. Electr. Eng. Informatics, 4. J. Yuan and C. Svensson, "High-speed cmos circuit technique," High-Performance Syst. Des.Circuits Log